

ERP Integration Report:

A Live Odoo ERP Auction Pipeline built on the Gould & Bonart (2015) “Queue Imbalance as a One-Tick-Ahead Price Predictor in a Limit-Order-Book”

Vipin V Lokesh
vipin.lokesh22@gmail.com
PES University

Summary

This report documents a live, end-to-end auction workflow that takes in real bids and asks from a role-gated React front-end. The bids are then run through an IMM regime-switching Kalman filter, validated against the FI-2010 benchmark, after which the resulting auction outcome is executed as a real record in an Odoo ERP instance – CRM leads, Sales Orders, posted customer invoices, and registered payments.

The pipeline is organised under eight logical stages, implemented across nine Python modules, a Flask API, and a local JSON ledger for states that Odoo has no native concept of. A design decision that recurs throughout and is treated as central here is that in several places where the live data’s structure differs greatly from the FI-2010’s (single-level ask book, a rolling bid-resting window instead of continuous two-sided replenishment) are called out explicitly as by-design limitations, not bugs, so that a number from one layer is never mistaken for ground truth in another.

The underlying signal that this pipeline is built to act on originates with Gould & Bonart (2015), “Queue Imbalance as a One-Tick-Ahead Price Predictor in a Limit-Order-Book”: the relative depth of resting bid versus ask volume at the best quotes, used as a short-horizon directional signal. `LOB_ssm.py` computes that same queue imbalance quantity from live bid/ask state, then passes it through the Kalman/ IMM filter before reaching the diagnostics endpoint this pipeline exposes. One caveat that must be noted is that the live diagnostics provided here should be accordingly read as a smoothed, lower-variance view of queue imbalance rather than as a validated improvement on Gould & Bonart’s original predictor.

1. Pipeline Architecture

This pipeline runs in eight stages, most of them corresponding one-to-one with a Python module in the repository, which itself corresponds with the Odoo database via various Python dependencies for the same.

Sl No.	Stage	Module	Function
1.	Ingest	bid_ingestion.py	Validates tick size and market hours; Registers every valid bid as a CRM lead whether it wins or loses.
2.	Analysis and Market State Construction	LOB_ssm.py (run_on_odoo_bids_and_asks)	Reconstructs a tick stream from Odoo bids + Askbook events; runs the IMM/Kalman Filter and reports the regime probabilities, queue imbalance, and other diagnostics.
3.	Auction	LOB_ssm.py (find_highest_bid_from_odoo)	Re-reads the bid stream from stage 2. Applies price-time priority against the current ask price (subject to change based on seller's decision) to determine winner(s).
4.	Promote	promote_winners.py	Converts each winning lead into an opportunity; creates and confirms a Sales Order.
5.	Deplete	ask_book.py	Reflects the fill against the ask book so remaining depth is accurate for next cycle.
6.	Ledger	Blanket_order_ledger.py	Tracks shares committed/sold/remaining per stock's blanket order.
7.	Settle	Accounting.py Settle_existing.py	Generates and posts a customer invoice, registers payment.
8.	Inventory	run_pipeline.py	Shares_remaining > 0 → Loop the same stock; otherwise, advance to the next stock in the queue

Table 1: The eight pipeline stages, their implementing module and their function

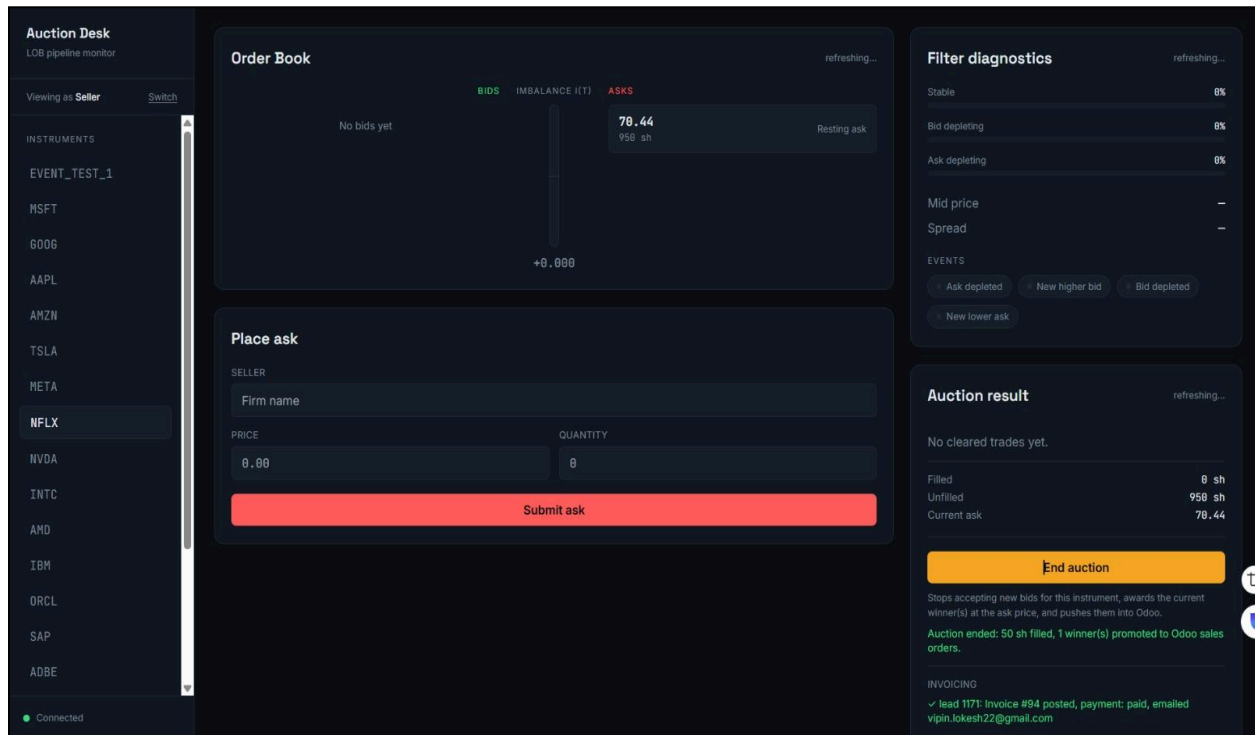
1.1. End-to-end data flow

```

Buyer/Seller (React frontend, role-gated via RoleGate.jsx)
|
v
Flask backend (Test_bid_generation.py)
|
+- POST /bids → bid_ingestion.py → Odoo CRM (crm.lead)
+- POST /asks → ask_book.py (local JSON, single best level)
+- GET /diagnostics/:id → LOB_ssm.py (IMM filter, live ticks)
+- GET /auction/:id/winner → LOB_ssm.py (read-only preview)
+- POST /auction/:id/end
    |
    +- LOB_ssm.find_highest_bid_from_odoo (determine winner(s))
    +- promote_winners.py --> Odoo (lead → opportunity → confirmed
sale.order)
    + - ask_book.deplete() (reflect the fill)

```

2. Frontend and API surface



2.1. React frontend:

- RoleGate.jsx – resolves buyer/seller from a role parameter or session storage, prompting with a picker if neither is set. Role is held per browser tab, not shared globally, so two tabs (or two people) can hold different roles concurrently.
- OrderForm.jsx – locks the order entry form to bid-only or ask-only depending on the active role.
- AuctionPanel.jsx – a polled, live auction preview plus an “end auction” action, visible only to seller/both roles.
- App.jsx – Polls bids, asks, diagnostics and auction ID winner every 4 seconds. Once a role is chosen, it also manages dynamic instrument allocation via a ?stock= parameter and a sidebar control.
- Client.js – a mock mode environment flag switches between canned mock responses and real Flask endpoints so that the frontend can be developed and demoed without a live Odoo connection.

2.2 Backend routes within Test_bid_generation.py

No.	Route	Method	Behaviour
1.	/bids	GET / POST	List resting bids for a stock, or submit a new bid.
2.	/asks	GET / POST	Current best ask price, or post/lower ask price
3.	/diagnostics/:stock_id	GET	Latest IMM regime change probabilities, queue imbalance and deterministic event flags
4.	/auction/:stock_id/winner	GET	Read-only preview of the would-be winner(s), makes no Odoo writes, used mainly to keep the poll updated
5.	/auction/:stock_id/end	POST	Executes the auction, rechecks the Live ask (optionally verifies against a client-supplied expected_ask_price and returns HTTP 409 if it has moved), determines winners, promotes them, and depletes the ask book.

Table 2: Flask API surface exposed to the React Frontend

3. Odoo Integration

3.1 Connection and Access:

All Odoo access goes through the standard XML-RPC endpoints, configured via an `OdooConfig` dataclass (`url`, `db`, `username`, `api_key`). Credentials are read from environment variables on render at import time, so the same code path works both under local development and behind a production WSGI server such as `unicorn`, where environment variables are the only practical way to inject configuration.

3.2. Stage 1 – Bid Ingestion:

- `bid_ingestion.py` functions as the primary entry point, recording every validated bid as a CRM lead record regardless of the eventual auction outcome.
- Before registration, two validation gates are applied: `validate_tick_size()`—which typically operates in `round_to_valid` mode to snap off-tick prices to the nearest increment, mirroring exchange-side normalisation—and a market-hours gate that rejects submissions when the session is closed.
- Each lead is initialised as a non-opportunity CRM lead, with a description field containing a deterministic bid reference used by `Bid_IngestionClient.bid_already_registered()` for idempotency. The module also manages res. partner records, ensuring buyer email synchronisation across subsequent bids.
- Stage 1 does not consult the SSM at all. The two stages are treated as two separate consumers of the same event stream, so a change to the filter’s logic can never silently affect what gets permanently recorded in CRM.

3.3. `LOB_ssm.py` explained:

Imports:

- `re` – Regular Expressions:
 - Utilised for defining complex pattern sequences for automated string matching.
 - Removes the necessity for manual string slicing when parsing structured data streams.
- `xmlrpc.client`:
 - Provides the interface between Python-based logic and the target Odoo environment.

- Facilitates remote procedure calls to manage state and execute records across systems.
- Datetime: Assigns temporal markers to incoming bids to enforce price-time priority during auction resolution.
- Dataclasses: Encapsulates bid attributes into structured objects, allowing for cleaner attribute access via dot notation.
- Numpy: Serves as the computational backbone for high-dimensional matrix operations required by the IMM/Kalman regime filter.

The computational framework for analysis utilises a unified estimation architecture that remains agnostic to the data source, processing both historical FI-2010 benchmarks and live ERP bid/ask streams with identical logic:

- **State vector:**

The system tracks a six-dimensional state defined as:

$$x = [best_bid, best_ask, bid_queue_size, ask_queue_size, bid_drift, ask_drift]$$

Here, the drift components serve as latent variables—inferred rather than observed—that quantify the underlying expansion or contraction rates of resting liquidity on either side of the book.

- **Transition matrix F:**

Implementing a local-trend model, this matrix maintains an identity structure except for off-diagonal terms –

$$F[q_b, mu_b] = dt$$

$$F[q_a, mu_a] = dt.$$

These elements integrate drift into queue depth at every tick (`build_transition_matrix()`). While F is invariant across regimes, specific market behaviours are captured via modifications to process noise, bias, and observation parameters.

- **Process noise and bias (`build_process_noise(regime)`):**

For each regime, the model generates a (Q, bias) pair used in the state transition –

$$x_t = Fx_{t-1} + bias + w_t$$

The STABLE regime assumes minimal noise ($base_q = 0.05$) and zero bias. In contrast,

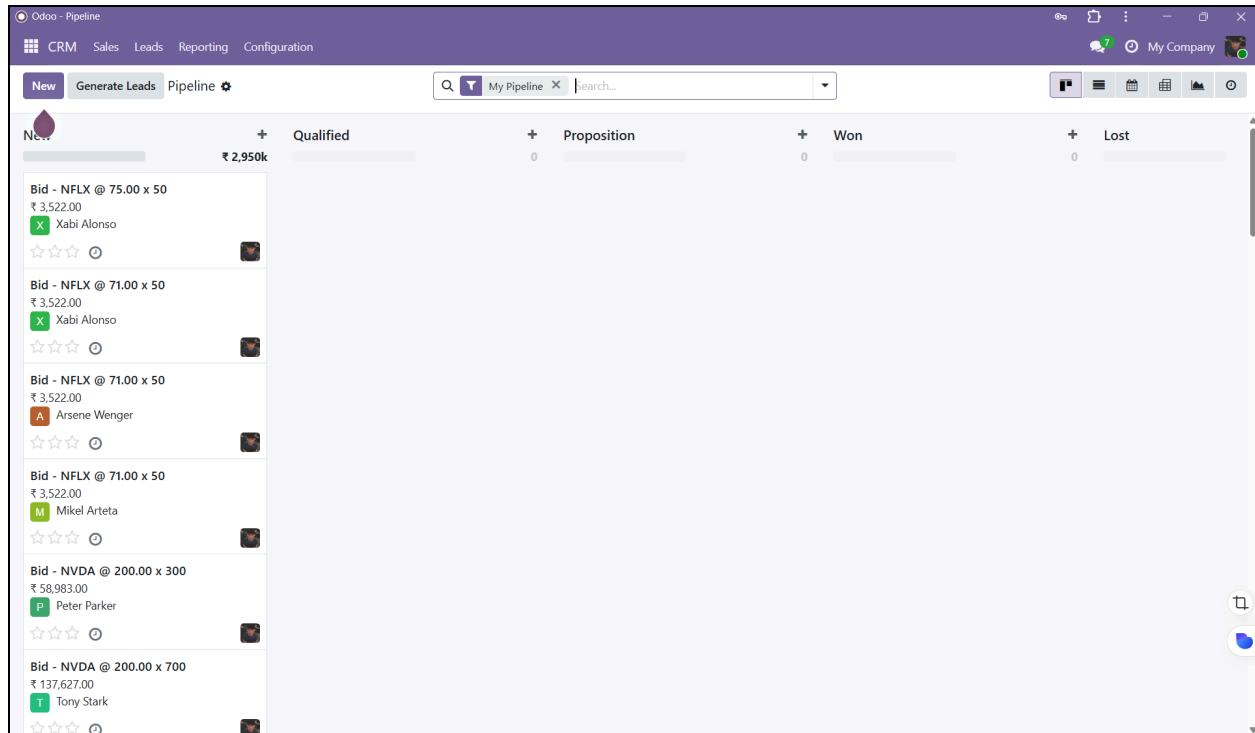
BID_DEPLETING inflates drift noise on μ_b by $\sim 40\times$ ($\text{depletion_q} = 2.0$) and introduces a direct shock to q_b ($\text{queue_shock_q} = 0.5$) while applying a negative $\text{depletion_bias} = -3.0$ to signal anticipated volume loss. ASK_DEPLETING operates symmetrically.

- Observation noise (`build_observation_noise(regime)`):**
 To ensure the filter remains responsive during rapid depletions, the observation noise for the affected queue is reduced to 15% of its baseline. This adjustment increases the Kalman gain, compelling the filter to prioritise incoming data points over model-driven smoothing.
- Single-regime filter step (`kf_predict`, `kf_update`):**
 The prediction phase advances the state and covariance ($P_{pred} = F P F^T + Q$); subsequently, the update incorporates the newest observation $z = [b, a, q_b, q_a]$ using the gain $K = P H^T S^{-1}$. This step also calculates the Gaussian log-likelihood, which the IMM layer utilises to dynamically evaluate the fitness of each regime.
- Mixing:** Before each tick, the initial state for every regime is formed from a weighted blend of the previous posteriors. This transition uses a sticky probability matrix Π (diagonal entries 0.82), ensuring that state estimates retain memory of the most likely preceding regime.
- Mode-matched filtering:** The `kf_predict` and `kf_update` cycles proceed in parallel across all three regimes. Each utilises its own unique F , Q , bias, and R parameters, yielding three distinct posterior candidates and their associated log-likelihoods.
- Regime-probability reweighting:** A posterior probability vector μ is derived via Bayes' rule. Regimes that predicted the actual tick observation with greater accuracy (minimal innovation) gain dominance. This vector is reset each tick, preventing the accumulation of historical bias.
- Combination:** The resultant `imm.combined_state` constitutes a μ -weighted mixture of the parallel posteriors. This provides a singular, stabilised estimate for downstream consumption while maintaining multi-regime tracking internally.
- Feature extraction (`extract_features()`):** This phase flattens the `combined_state` and μ into a feature dictionary. It exports mid-prices, spreads, `queue_imbalance`, the composite drift_signal ($\mu_b - \mu_a$), and the discrete regime probabilities (`p_stable`, `p_bid_depleting`, `p_ask_depleting`).
- Relationship to EventFlags:** These probabilistic metrics serve as soft indicators that complement the pipeline's deterministic EventFlags (via `detect_events()`). While the filter provides a smooth

likelihood landscape, the EventFlags enforce binary, rule-based logic to detect structural shifts like zero-depth exhaustion or price improvement events.

3.4 Stages 2–4 — analysis, decision, and promotion

- **Stage 2 (analysis):**
 - Rebuilds a tick-by-tick view of best bid, best ask, and bid/ask quantities by combining the recorded bid stream with the logged ask events in chronological order, then runs that stream through the same regime-switching filter used in the FI-2010 evaluation.
 - The output — regime probabilities, queue imbalance, and four deterministic flags (ask depleted, new higher bid, bid depleted, new lower ask) — is purely informational at this stage and does not influence the winner determination in Stage 3.
- **Stage 3 (decision):**
 - Independently re-reads the bid stream and applies standard price-time priority: among bids at or above the current ask price, the highest bid wins first, ties broken by earliest timestamp; fills are allocated against the ask quantity until it's exhausted, so a single ask can be split across multiple winning bids.
 - Winners are awarded at the ask price, not their own bid — standard call-auction convention. Bids below the ask price never win; they remain on record as a permanent trace that they existed and didn't clear.
- **Stage 4 (promotion):**
 - Converts each winning bid into a confirmed sale: the winning lead is upgraded from a raw lead to an opportunity, a sellable product record is created for the stock if one doesn't already exist, a sales order is created and linked to that opportunity, and the order is confirmed. Losing bids are left untouched as unconverted leads.



3.5. Stage 5 – Ask book depletion (outside Odoo, by design)

- Asks are tracked in a local JSON file (ask_book.py), deliberately outside Odoo CRM: per the pipeline's original design, an ask represents an existing sell listing, a structurally different kind of data from an incoming buyer bid, and is modelled as a simple timestamped event log rather than a CRM object.
- AskBook supports posting a new sell limit order (which becomes the new best ask if its price is strictly better, or adds depth if it matches the current best price) and depleting the ask (reducing quantity, e.g. following a fill, and flagging full depletion when quantity reaches zero).
- AskBook tracks only a single best price/quantity level per stock — there is no seller identity and no depth beyond the top level, a limitation the repository documents explicitly rather than silently.

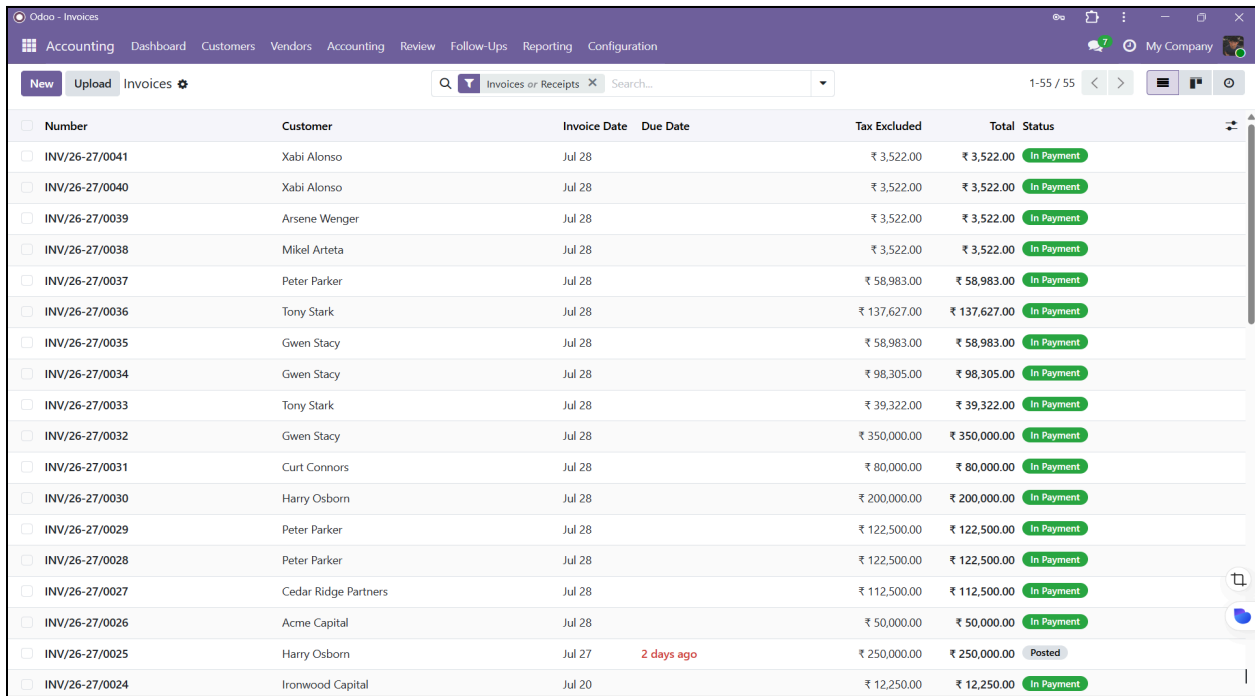
3.6. Blanket Orders (blanket-order-ledger)

- **What a blanket order is.** A standing agreement to sell a fixed total quantity of a stock across multiple auction cycles, rather than in one shot — the pipeline runs repeated auctions against the

same stock until that total is filled.

- **Local ledger, not Odoo fields.** Shares committed, sold, and remaining are tracked in a local JSON ledger rather than as custom Odoo fields, since "shares committed/sold/remaining" has no native Odoo concept, and adding it via custom fields is an extra, install-specific layer of configuration that's easy to get wrong through the API — a local ledger is more robust and easier to inspect directly.
- **Drives the loop/advance decision.** The ledger's remaining-shares value determines whether the pipeline loops the auction back to the same stock (shares remain) or advances to the next stock in the queue (the blanket order is filled).

3.7. Accounting and settlement:



Number	Customer	Invoice Date	Due Date	Tax Excluded	Total	Status
INV/26-27/0041	Xabi Alonso	Jul 28		₹ 3,522.00	₹ 3,522.00	In Payment
INV/26-27/0040	Xabi Alonso	Jul 28		₹ 3,522.00	₹ 3,522.00	In Payment
INV/26-27/0039	Arsenie Wenger	Jul 28		₹ 3,522.00	₹ 3,522.00	In Payment
INV/26-27/0038	Mikel Arteta	Jul 28		₹ 3,522.00	₹ 3,522.00	In Payment
INV/26-27/0037	Peter Parker	Jul 28		₹ 58,983.00	₹ 58,983.00	In Payment
INV/26-27/0036	Tony Stark	Jul 28		₹ 137,627.00	₹ 137,627.00	In Payment
INV/26-27/0035	Gwen Stacy	Jul 28		₹ 58,983.00	₹ 58,983.00	In Payment
INV/26-27/0034	Gwen Stacy	Jul 28		₹ 98,305.00	₹ 98,305.00	In Payment
INV/26-27/0033	Tony Stark	Jul 28		₹ 39,322.00	₹ 39,322.00	In Payment
INV/26-27/0032	Gwen Stacy	Jul 28		₹ 350,000.00	₹ 350,000.00	In Payment
INV/26-27/0031	Curt Connors	Jul 28		₹ 80,000.00	₹ 80,000.00	In Payment
INV/26-27/0030	Harry Osborn	Jul 28		₹ 200,000.00	₹ 200,000.00	In Payment
INV/26-27/0029	Peter Parker	Jul 28		₹ 122,500.00	₹ 122,500.00	In Payment
INV/26-27/0028	Peter Parker	Jul 28		₹ 122,500.00	₹ 122,500.00	In Payment
INV/26-27/0027	Cedar Ridge Partners	Jul 28		₹ 112,500.00	₹ 112,500.00	In Payment
INV/26-27/0026	Acme Capital	Jul 28		₹ 50,000.00	₹ 50,000.00	In Payment
INV/26-27/0025	Harry Osborn	Jul 27	2 days ago	₹ 250,000.00	₹ 250,000.00	Posted
INV/26-27/0024	Ironwood Capital	Jul 20		₹ 12,250.00	₹ 12,250.00	In Payment

- **Settlement flow.** A confirmed sale order is settled in three steps: (1) a draft customer invoice is generated via the same wizard the "Create Invoice" UI button drives, used deliberately since Odoo 17+ blocks calling the private invoice-creation method directly; (2) the invoice is posted (draft → posted); (3) a payment is registered against it via Odoo's payment-registration wizard. Odoo's accounting engine handles the resulting journal postings internally — no manual journal-entry step in this pipeline.

- **Most install-specific stage.** Required taxes, fiscal positions, chart-of-accounts structure, and default payment journals all vary per Odoo install, and this stage makes minimal assumptions (no tax, default journal) that are more likely than CRM/Sales to need tuning against a specific real instance.
- **Manual trigger, on purpose.** Invoicing is never auto-triggered when an auction ends — it's called as an explicit, separate phase, so a failed settlement can be re-run and diagnosed against a known sale order without re-running the whole pipeline (and generating a fresh batch of duplicate test bids just to get back to that point).
- **Visibility into pending settlements.** A separate utility lists every sale order still awaiting invoicing directly from Odoo, without clicking through the UI.

4. Orchestration (`run_pipeline.py`)

`run_pipeline.py` drives the full loop across a configured queue of stocks, implementing Stage 8's continuation logic end to end:

```
Phase 1 - INGEST:  generate/receive bids, validate, write to Odoo.

Phase 2 - ANALYZE: read the same bids back from Odoo, run the SSM.

Phase 3 - DECIDE:  read the same bids again, determine the auction
                  winner against a given ask price/qty.

Phase 4 - PROMOTE: promote winners -> opportunities -> confirmed sales
                  orders. Promoted winners become type="opportunity"
                  and drop out of future Phase 3 queries automatically,
                  so a later cycle never re-awards an already-won bid.

Phase 5 - SETTLE:  invoice + post + register payment (Accounting.py).

Phase 6 - LEDGER:  update the blanket order's committed/sold/remaining
                  shares (local ledger, not Odoo itself).

Phase 8 - CONTINUE: shares_remaining > 0 -> loop back to Phase 1 for
                  the SAME stock. shares_remaining <= 0 -> the
                  blanket order is fulfilled; move to the next stock.
```

Figure 2. `run_pipeline.py`'s phase structure, reproduced from its module docstring.

- **Per-stock cycle cap.** Each stock is capped at MAX_CYCLES_PER_STOCK (5 by default) so a stock receiving no clearing bids can't loop indefinitely; once the cap is hit, the pipeline logs that the blanket order wasn't filled and moves on to the next stock in the queue.
- **Ask price sourcing.** Each cycle's ask price comes from the live ask book if one has been posted for that stock; only if no ask has ever been posted does it fall back to a 7-day average close pulled from Finnhub — real, previously-posted seller intent is always preferred over an external reference price.
- **Completion email.** Once the full stock queue finishes, a plain-text confirmation email is sent through Odoo's own outgoing mail server, summarising the stocks processed and the number of invoices settled during the run.
- **Invoice emails.** Posted invoices are additionally emailed individually as a direct link into Odoo rather than a rendered PDF attachment, avoiding a dependency on Odoo's report-rendering controller behaving consistently across versions.

Xabi Alonso

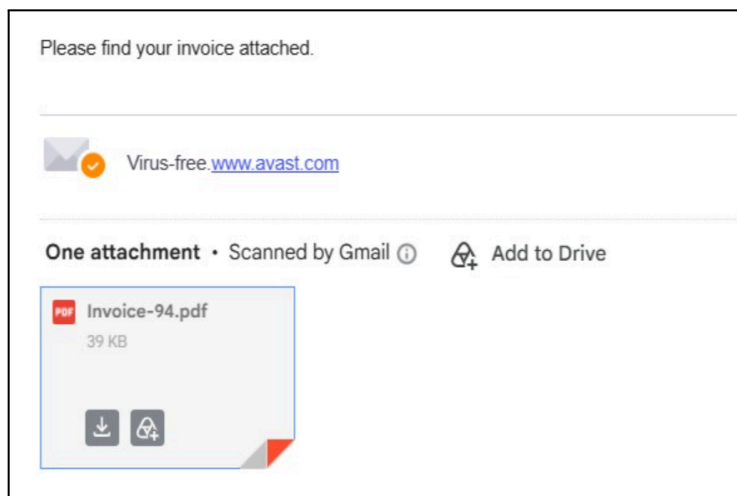
Invoice INV/26-27/0041

Invoice Date
07/28/2026
Due Date
07/28/2026
Source
S00083
Reference
S00083

Description	Quantity	Unit Price	Amount
Auction stock - NFLX 50.00	70.44		₹ 3,522.00
Total			₹ 3,522.00
<i>Paid on 07/28/2026</i>			₹ 3,522.00
Amount Due			₹ 0.00

Total amount in words:
Three Thousand Five Hundred And Twenty-Two Rupees

Payment Communication: INV/26-27/0041



5. Known limitations (by design)

The repository's own documentation is explicit that the following are intentional scope boundaries rather than oversights — reproduced here because they materially affect how any number produced by this layer should be interpreted, especially relative to the FI-2010 diagnostic layer covered in the companion report:

- Ask book is single-level. AskBook tracks one best price/quantity per stock, with no seller identity and no depth beyond the top level; GET /asks always returns at most one row.
- Bid queue depth is a rolling snapshot, not a lifetime total. `best_bid_qty` only counts bids within a trailing `BID_RESTING_WINDOW_SECONDS` (default 600s / 10 minutes) window. Without this window, depth could only ever accumulate — nothing removes a bid from the running sum except it winning an auction — which would drag queue imbalance toward an extreme rather than the balanced range FI-2010 exhibits (≈ 0.53 – 0.58 , per the repository README); the window should be tuned if the platform's real bidding cadence differs materially from the default.
- Invoicing is manual. Ending an auction promotes winners to a confirmed Odoo sale order; it does not by itself invoice or collect payment. `Settle_existing.py` (or `Accounting.py` directly) must be run as a separate, explicit step.
- No seller ownership of a listing. Any user can post a better ask and effectively take over as “the” seller for an instrument; there is no listing-ownership concept comparable to, e.g., a marketplace platform's seller accounts.
- Tick size and market hours are placeholder-real. `finnhub_market_data.py` uses Finnhub's free tier: tick size is hardcoded at 0.01, and while market-open status is a genuine live API call, quote depth (`best_bid_qty` / `best_ask_qty` from `get_reference_quote`) is synthetic rather than real order-book depth.

The README is explicit on one further point worth restating here: the ≈ 0.53 – 0.58 queue-imbalance range validated against FI-2010 (companion report, §4) is a sanity check on the filter's math, using real, continuously two-sided-replenished exchange order flow — it is not a number the live pipeline should be expected to reproduce, since live test data does not yet have equivalent two-sided replenishment on both sides of the book at once. Treating a live-pipeline queue-imbalance reading as “wrong” because it doesn't match the FI-2010 reference range would conflate two structurally different data sources that happen to share one estimation engine.

6. Deployment

No.	Component	Target	Notes
1.	Frontend	Vercel	Static hosting; free tier sufficient.
2.	Backend	Render	Not serverless – needs a long-running process. Uses a Procfile (gunicorn – workers1 –threads 8 –timeout 120 Test_bid_generation: app) and requirements.txt
3.	Odoo	Hosted (Odoo.sh) Locally downloaded and run alongside a tunnel (ngrok)	A local Odoo reached via tunnel is a fragile link for anything beyond demos, since it depends on the developer's machine staying online.
4.	CORS	Configured in Test_bid_generation.py	Must list the real deployed frontend origin in CORS(app, origins=[...]).

Table 3. Deployment targets, reproduced from the repository README / DEPLOY.md